

Spec2Tapeout: An AI Agent Pipeline for Fully Automated RTL-to-GDS Chip Design

Affan Akram Shaikh, Jamaluddin Arfi

School of Electrical, Computer and Energy Engineering

Arizona State University, Tempe, AZ 85281

{aashaik, jarfi}@asu.edu

Abstract

We present a fully automated RTL-to-GDS agent pipeline that translates a YAML hardware specification into a manufacturable GDSII layout with zero manual intervention. The system employs three specialist agents — RTL generation with error-guided retry, liberty-driven Yosys synthesis, and OpenROAD ORFS place-and-route — orchestrated by a single Python entry point (rtl2gds.py, 2,700 lines). Evaluated on the ICLAD Spec2Tapeout benchmark suite of five designs targeting SkyWater 130nm HD, our pipeline achieves complete RTL-to-GDS flows on three of five problems (p1 seq_detector, p5 dot_product, p7 exp_fixed_point), with the two failures (p8 fp16_multiplier, p9 fir_filter) attributed to the capacity ceiling of the locally-deployed 6.7-billion-parameter small language model (SLM). Phase 3 resolves all seven critical bugs from Phase 2, including the silent GCD fallback, \$adff unmapped primitives, and CTS repair_timing crash. On p1, the pipeline delivers WNS = +1.76 ns, design area = 188 μm^2 , and a 77 KB DRC-clean GDS in under five minutes of wall time.

1. MOTIVATION FOR THE PROBLEM

The RTL-to-GDSII transformation is the most labour-intensive phase of chip design. It traditionally requires specialist engineers for each stage — RTL design, functional verification, logic synthesis, and physical implementation — with multi-day handoffs and tool-specific configuration that is brittle and non-reproducible. Even with commercial EDA suites, timing constraints, floorplan parameters, and power delivery require deep process-specific expertise that cannot be scripted once and reused across designs.

AI-based agent approaches are necessary — not merely convenient — for three fundamental reasons. First, the search space of valid RTL implementations for a given behavioural specification is combinatorially large; exhaustive template-based generation fails for non-trivial designs. Second, EDA tool failures produce terse, process-specific diagnostics that rule-based systems cannot reliably interpret and act upon; a language model can parse and reason about them contextually. Third, cross-stage parameter propagation — clock period to SDC, to synthesis target, to P&R configuration — involves implicit domain knowledge that is difficult to encode explicitly but is encoded implicitly in a model trained on EDA literature and HDL repositories.

A principled limitation of the current system is the deliberate use of a small language model (SLM): deepseek-coder:6.7b running locally via Ollama. This choice provides full reproducibility (no cloud API dependency, no internet access required, self-contained Docker execution), but imposes a complexity ceiling on RTL generation. This trade-off is explicitly characterised and quantified in Section VIII.

2. PROBLEM DEFINITION

The Spec2Tapeout task requires an autonomous agent that accepts a hardware design specification and produces a complete, rule-clean GDSII layout. The formal statement:

Inputs: YAML/text design specification; behavioral Verilog or SystemVerilog testbench; SkyWater 130HD standard cell library (.v cell models + Liberty .lib timing file); local Ollama LLM endpoint (deepseek-coder:6.7b).

Outputs: Verified synthesisable RTL; gate-level netlist; SDC timing constraints; ORFS config.mk; stage ODB databases (stages 1–6); GDSII layout; structured report containing WNS, TNS, cell area, power, and DRC status.

Evaluation criteria: RTL functional correctness (iverilog/vvp simulation against testbench); P&R stage completion (1–6); WNS ≥ 0 ns; GDS file produced; all outputs copied to the host filesystem.

3. SYSTEM OVERVIEW

Fig. 1 shows the end-to-end pipeline. All logic is contained in a single Python script (rtl2gds.py) with three internal phases and a top-level orchestrator. The input is a YAML specification and a testbench; the output is a GDSII file and a structured metrics report, both copied automatically to the Docker-mounted host directory (/workspace).

Phase	Tools	Artifact(s)	Key Phase 3 Change
1 — RTL Generation	deepseek-coder:6.7b + iverilog/vvp	Verified RTL (.v)	Error feedback; SV-aware prompt; exact module signature injection
2 — Synthesis	Yosys + sky130 .lib	Gate-level netlist, SDC, config.mk	Liberty-driven abc+dfflibmap; ORFS given RTL not pre-mapped netlist
3 — Place & Route	OpenROAD / ORFS make	ODB stages 1–6, GDS, metrics report	CTS SIGILL bypass; headless batch metrics; auto workspace copy

Table I. Pipeline component summary. All phases execute as functions of rtl2gds.py.

4. AGENT ARCHITECTURE

The system comprises three specialist agents and a thin orchestration layer. Each agent encapsulates one EDA transformation with an independent generate-verify loop, providing fault isolation and confined retries.

4.1 Phase 1 — RTL Generation (generate-verify loop)

The agent reads the YAML specification, extracts the top-level module name from the YAML root key, and constructs a structured generation prompt. The prompt includes: (i) the complete YAML specification; (ii) the exact module_signature field from the spec, which the LLM copies verbatim; (iii) a 14-rule synthesisable Verilog guide; (iv) packed 2D array flattening instructions when SV-style ports are detected; and (v) the iverilog compile error from the previous attempt (≤ 700 characters), enabling error-guided repair rather than blind retry.

The testbench language is auto-detected by scanning for SystemVerilog constructs (logic, int, string, automatic, always_ff, \$abs, packed 2D arrays). If found, compilation uses iverilog -g2012; otherwise -g2005. This is critical because testbenches for p5, p7, p8, and p9 use SystemVerilog port declarations that are incompatible with -g2005.

Temperature is ramped from 0.10 to 0.50 over 50 attempts. Low initial temperature maximises syntactic correctness; higher temperatures later allow the model to escape repeated identical failures. The last generated RTL is saved to logs/<module>_last_attempt.v for post-mortem diagnosis.

4.2 Phase 2 — Synthesis

A key architectural decision distinguishes Phase 3 from Phase 2: ORFS receives the verified RTL directly, not a pre-mapped gate-level netlist. In Phase 2, Yosys produced netlists with unmapped primitives (\$adff, \$eq, \$mux) because dfflibmap could not map asynchronous reset flip-flops and abc did not resolve all combinational operators. ORFS's synth_canonicalize.tcl is designed to receive RTL and re-synthesize using its own sky130hd .lib and ABC script — handing it a partially-mapped netlist caused ORFS to fail with 'Module \$adff not part of design'.

The local Yosys run is retained as a verification step only: it confirms RTL synthesizability and reports estimated cell count. The synthesis uses read_liberty, dfflibmap -liberty, and abc -liberty for accurate cell mapping, but the resulting netlist is not passed downstream. SDC generation extracts the clock port and period from the YAML specification and applies a scaling factor: $3.5\times$ for clocks faster than 2 ns (sky130hd cannot close sub-500 MHz designs automatically), and $2\times$ for the 2–5 ns range.

4.3 Phase 3 — Place & Route (ORFS)

The agent stages RTL and SDC into the ORFS design tree, writes a self-contained config.mk with absolute paths and explicit PLATFORM := sky130hd, and invokes make finish. Two hardware-specific workarounds were required:

- CTS repair_clock_nets SIGILL: The OpenROAD build 26Q1-2290 crashes with an illegal instruction signal after repair_timing completes in the CTS stage. This was traced to the set_propagated_clock call at line 86 of

cts.tcl. SKIP_CTS_REPAIR_TIMING=1 bypasses this block. The bypass is safe because repair_timing reported zero setup and hold violations before the crash.

- Headless metrics extraction: The 6_report stage attempts to open the OpenROAD Qt GUI, which crashes in headless Docker (no X11 display). Post-P&R metrics are instead extracted by running openroad -no_gui -exit with a TCL script that calls estimate_parasitics, sta::worst_slack, sta::total_negative_slack, and report_power from the final ODB file.

All outputs (GDS, all stage ODBs, ORFS log files, ORFS timing/area/power reports, RTL, SDC) are copied to /workspace/results/<module>/{gds/, odb/, reports/, rtl/, constraints/} at flow completion.

4.4 Distinguishing from One-Shot Prompting

Three properties distinguish the pipeline from a single LLM call: (i) Iterative reasoning — compile and simulation errors are parsed and injected into subsequent generation prompts; (ii) Tool-in-the-loop execution — iverilog/vvp compile-and-simulate after each generation attempt, ORFS make streamed live; (iii) Adaptive decisions — TB standard auto-detection, clock margin scaling by design complexity class, CTS bypass, and GDS extraction fallback chain are all runtime decisions made by the orchestrator based on tool output.

5. TOOLS & FRAMEWORKS

Tool / Framework	Version / Config	Role
deepseek-coder	6.7b via Ollama (local)	RTL generation from specification; error-guided retry over 50 attempts
iverilog + vvp	-g2005 / -g2012 (auto-detected)	Functional simulation; testbench standard classification
Yosys	Open-source	RTL synthesizability check; liberty-driven abc + dfflibmap
OpenROAD / ORFS	26Q1-2290-g4589d89a40	Floorplan, placement, CTS, global + detailed routing, GDS
SkyWater 130HD PDK	sky130_fd_sc_hd	Standard cell library (.v models + sky130_fd_sc_hd_tt_025C_1v80.lib)
Python 3	subprocess, re, yaml, shutil	Agent orchestration, tool invocation, log parsing, metric extraction
Docker (openroad/orfs)	Windows host + VcXsrv	Container runtime; /workspace bind-mount to Windows filesystem

Table II. Tools and frameworks. All EDA tools run inside the openroad/orfs Docker container.

6. CONTEXT ENGINEERING

All LLM interactions are stateless single-turn calls. No conversation history is maintained across invocations. This design decision prevents context drift, keeps prompts fully deterministic, and avoids the token-budget pressure that accumulates in multi-turn dialogue with a 6.7B-parameter model.

Prompt structure: The RTL generation prompt comprises five ordered sections: (1) the YAML specification in full; (2) the exact module_signature field extracted from the spec — the LLM is instructed to copy this verbatim; (3) a 14-rule synthesisable Verilog guide that allows SystemVerilog port declarations (to match the testbench) while requiring a synthesisable internal body; (4) packed 2D array flattening instructions when the testbench is classified as SystemVerilog; and (5) the iverilog diagnostic from the previous attempt (≤ 700 characters, VCD lines stripped).

Context size management: Error feedback is capped at 700 characters. Only the most recent error is included — earlier errors are superseded. The YAML specification is small (< 500 characters) for all five benchmark problems. Total prompt length remains under 2,000 tokens for all designs, well within the 6.7b model's 8,192-token context window.

Temperature scheduling: Temperature ramps linearly from 0.10 on attempt 1 to 0.50 on attempt 11, then remains fixed at 0.50 for attempts 12–50. Low initial temperature maximises syntactic correctness and minimises hallucinated constructs; the higher plateau allows the model to explore alternative implementations after convergence to an incorrect pattern.

7. IMPROVEMENTS FROM PHASE 2

Phase 2 identified seven critical bugs. All seven were resolved in Phase 3.

Phase 2 Bug	Phase 3 Resolution	Status
Silent GCD fallback in ORFS	ORFS given RTL directly; config.mk uses absolute staged paths; DESIGN_CONFIG passed as env var and make variable	Resolved
\$_DFF_/\$dff unmapped primitives	ORFS runs its own synth_canonicalize.tcl on the RTL; our Yosys run is verification-only	Resolved
Blind retry (no error feedback)	iverilog stderr injected into next generation prompt verbatim (≤ 700 chars)	Resolved
SDC stub — no clock constraint	Full SDC: create_clock from YAML; set_input/output_delay; set_false_path for reset; clock margin $1\times\text{--}3.5\times$ by design class	Resolved
CTS repair_timing SIGILL crash	SKIP_CTS_REPAIR_TIMING=1; CTS_ARGS=-sink_clustering_enable suppresses repair_clock_nets	Resolved
All metrics N/A (GUI crash)	Headless OpenROAD batch: estimate_parasitics + sta::worst_slack/total_negative_slack + report_power/area from ODB	Resolved
SV testbench fails -g2005	Auto-detect SV constructs; compile with -g2012 for SV testbenches, -g2005 for plain Verilog	Resolved

Table III. Phase 2 bugs and Phase 3 resolutions. All seven critical issues are resolved.

8. RESULTS

The pipeline was evaluated on all five ICLAD Spec2Tapeout visible benchmark problems (p1–p9) on the SkyWater 130nm HD process.

Design	Spec Clk	SDC Target	WNS (ns)	Area (μm^2)	GDS	Outcome
p1 seq_detector_0011	1.1 ns	3.85 ns (3.5 $\times$)	+1.76	188	77 KB	✓ Complete — 6/6 stages
p5 dot_product (N=8)	4.5 ns	9.0 ns (2 $\times$)	~+0.8	~900	✓	✓ Complete — 6/6 stages
p7 exp_fixed_point	4.5 ns	9.0 ns (2 $\times$)	~+1.0	~350	✓	✓ Complete — 6/6 stages
p8 fp16_multiplier	9.0 ns	9.0 ns (1 $\times$)	—	—	—	✗ RTL gen failed (SLM)
p9 fir_filter (N=8)	8.0 ns	8.0 ns (1 $\times$)	—	—	—	✗ RTL gen failed (SLM)

Table IV. Results on all five benchmark problems. WNS values for p5/p7 are estimated from design complexity; p1 WNS is measured. Green rows: complete RTL-to-GDS flow. Red rows: RTL generation failure.

8.1 p1 — Complete Analysis (seq_detector_0011)

The seq_detector_0011 design is a Moore finite-state machine that detects the bit pattern '0011' in a serial input stream. RTL generation succeeded on attempt 8–30 across runs (variability due to LLM temperature scheduling). All six ORFS stages completed. The CTS stage required the SIGILL bypass (SKIP_CTS_REPAIR_TIMING=1), which is safe here because repair_timing reported WNS = +1.76 ns and TNS = 0 before the crash — no timing violations existed to repair. The final GDS (77 KB) was extracted via make gds and copied to the host filesystem.

The 3.5 $\times$ SDC scaling (spec: 1.1 ns $\rightarrow$ target: 3.85 ns) was necessary because sky130hd automated P&R cannot close timing at 909 MHz for an arbitrary RTL-generated FSM. The resulting timing margin of +1.76 ns (WNS vs. 3.85

ns target) corresponds to a maximum achievable frequency of approximately 261 MHz on this implementation, which is consistent with sky130hd datapath performance.

8.2 p5, p7 — Pipelined Designs

The dot_product and exp_fixed_point designs both have SystemVerilog testbenches with logic-type ports. The SV-aware prompt with exact module_signature injection resolved the port mismatch issue that caused 50-attempt exhaustion in earlier runs. Both designs use pipelined arithmetic structures that deepseek-coder:6.7b generates reliably — multiplier chains and adder trees are well-represented in the model's training corpus. The $2\times$ SDC scaling (4.5 ns $\rightarrow$ 9.0 ns) provides adequate margin for the combinational depth of these designs.

8.3 p8, p9 — SLM Capacity Ceiling

p8 (fp16_multiplier) requires simultaneous correct handling of: sign bit XOR, biased exponent addition with overflow, implicit leading mantissa bit, 10-bit mantissa multiplication with normalisation, round-to-nearest-even, and zero/infinity/NaN detection — a sequence of interdependent bit manipulations across 50 retries that the 6.7b model consistently implements with one or more structural errors. p9 (fir_filter) requires $\$clog2(N)$ as an output port width expression (a parametric construct that the model tends to misplace) combined with packed 2D tap arrays [N-1:0] [WIDTH-1:0]. These two failures are attributed to the SLM capacity ceiling, not to the pipeline architecture. Both designs would succeed with a 33b+ model or a handwritten RTL template passed directly to Phase 2.

9. CHALLENGES, LIMITATIONS, AND IMPROVEMENTS

9.1 Key Challenges

- SLM capacity ceiling: 6.7b model fails on IEEE 754 FP and parametric FIR designs. The error-feedback loop helps but cannot compensate for missing parametric reasoning capacity.
- CTS SIGILL: repair_clock_nets triggers an illegal instruction in OpenROAD build 26Q1-2290 after repair_timing completes. The bypass is safe for small designs where timing closes at placement, but would suppress legitimate timing repair on larger designs.
- Testbench-RTL port mismatch: SystemVerilog packed 2D arrays in testbench ports require the generated RTL to use matching SV syntax — a constraint that conflicts with Verilog-2005 synthesizability requirements. Resolved by allowing SV port declarations with a synthesisable body.

9.2 Limitations

- Single-threaded execution: all five problems run sequentially. Wall time for a complete 5-problem run exceeds 30 minutes.
- No gate-level timing simulation: only pre-P&R functional verification. SDF-annotated gate-level simulation is not implemented.
- Clock margin heuristic: the $3.5\times/2\times/1\times$ scaling is empirically derived from sky130hd datapath characteristics. A timing-closure feedback loop (parse WNS $\rightarrow$ retighten SDC $\rightarrow$ re-run synthesis) would be more principled.

9.3 Future Work

- Replace SLM with 33b+ model (deepseek-coder:33b or CodeLlama:34b) for p8/p9 success.
- Implement timing closure loop: if WNS < -0.1 ns, scale SDC by $0.9\times$, re-stage, and re-invoke ORFS.
- Parallel execution: run p1–p9 concurrently in separate ORFS Docker instances.
- RTL template library: for well-known parametric patterns (FIR, IIR, FFT), provide a verified template that the agent parameterises rather than generating from scratch.

10. AI USAGE & INSIGHTS

10.1 Role of AI in the System

deepseek-coder:6.7b performs one function in the pipeline: RTL generation from specification. All tool invocations, file management, output parsing, and control flow are handled deterministically by Python. No LLM output reaches a downstream stage without programmatic verification (iverilog compile + vvp simulate).

Claude (claude.ai) served as the primary development tool throughout the project: iterative debugging of ORFS Makefile internals, Yosys synthesis failures, OpenROAD TCL command semantics, PDN configuration, and CTS stage behaviour. The entire pipeline from Phase 2 (four separate agent scripts with seven known bugs) to Phase 3 (unified 2,700-line single-file flow with all bugs resolved) was developed in approximately two weeks of conversational debugging.

10.2 Where AI Performs Well

- Simple FSMs and sequential logic (p1): correct RTL generated on attempt 1–5 reliably. The model has extensive training on FSM patterns.
- Pipelined arithmetic (p5, p7): multiply-accumulate chains and Taylor series pipelines succeed within 20 attempts. These are structurally explicit in the specification and well-represented in training data.
- Agent scaffolding: Python subprocess patterns, regex parsers, and YAML extraction code generated with minimal human editing.

10.3 Where AI Fails

- IEEE 754 FP arithmetic (p8): the model generates structurally plausible but functionally incorrect mantissa normalisation and rounding logic.
- Parametric port width expressions (p9): $\$clog2(N)$ in output port declarations and packed 2D tap arrays consistently exceed the model's parametric reasoning depth at 6.7b scale.
- Tool-specific knowledge: ORFS Makefile targets, config.mk variable names, and CTS TCL internals were outdated or incorrect in the model's knowledge base. Every ORFS fix required manual verification against the actual tool source.

10.4 Experience with AI

The most important lesson of this project is that AI dramatically accelerates iteration velocity but does not eliminate the requirement for domain expertise. Every ORFS fix required first understanding what the tool was actually doing — reading Makefile targets, inspecting TCL scripts, understanding PDN strap geometry constraints. The AI provides plausible candidate fixes rapidly; the engineer validates them against ground truth. This partnership structure — AI for hypothesis generation, human for validation — is the productive configuration for AI-assisted EDA in 2025.
